Reaction Lens Systems: Reproducible Full-Catalog Retrieval of Reactome Reactions from Scientific Text

Imran Gadzhiev*

Independent Researcher, Manchester, United Kingdom

Abstract

Reaction Lens Systems is open-source research software that ranks Reactome reactions from English scientific titles and abstracts. A frozen scientific-text encoder and learned global/local scoring head evaluate all 15,692 reactions. The release provides pinned model artifacts, candidate-transformation checks, component timings and reproducible numerical evidence. In recorded CPU experiments, candidate transformations and blocking preserve sampled pairwise scores, while text encoding dominates complete prediction cost. The software supports inspectable retrieval proposals for human review and verification of released-model ranking metrics against recorded Reactome links.

Keywords: Scientific retrieval, Reactome, Research software, Reproducibility, Text mining

Required Metadata

Current code version

1. Motivation and significance

A researcher inspecting a scientific abstract may need to identify relevant biological reactions in a curated catalog. Reaction Lens Systems supplies ranked proposals for human review. Its engineering problem is maintaining an inspectable relationship between input text, catalog entries and predictions. Changing the order of candidates or the scoring block size should not silently change the score assigned to an unchanged article–reaction pair.
We study three engineering questions: which scores should remain stable when the catalog changes, how candidate blocking affects execution, and

*Corresponding author.
Email address: i.gadzhiev.m@gmail.com (Imran Gadzhiev)

Nr.	Code metadata description	Value
C1	Current code version	0.1.0.dev0; measured source revision `14c0ac048777` (full identifier in C2)
C2	Permanent GitHub link to code/repository used for this code version	GitHub: pinned source revision
C3	Legal Code License	MIT; third-party model and data terms documented separately
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python, PyTorch, NumPy; FastAPI/Uvicorn local service; Hugging Face artifact hosting
C6	Compilation requirements, operating environments & dependencies	Python 3.11–3.12; dependencies pinned in uv.lock; evaluated on macOS arm64 CPU FP32
C7	If available Link to developer documentation/manual	README: installation, usage and verification
C8	Support email for questions	i.gadzhiev.m@gmail.com

Table 1: Code metadata.

which component dominates local prediction time. The contributions are (1) an explicit execution contract derived from the global/local scorer and checked through candidate transformations, (2) measurements of exact blocked scoring and an unblocked reference, and (3) a reproducibility package linking aggregate results to recorded observations and a separate installation check. Together, these distinguish changes in rank caused by new competitors from changes in pairwise scores and locate the main cost of complete-text prediction.

2. Software description

2.1. Software architecture

Reactome supplies the biological reaction catalog [1]. SPECTER and SPECTER2 provide citation-informed scientific document representations [2, 3]. Reaction Lens Systems freezes its encoder and uses a learned global/local head. We evaluate a fixed checkpoint to study execution equivalence, component costs and inference reproducibility.

The processing path is English title/abstract, deterministic sentence spans, frozen text encoding, projected reaction vectors, exact blocked scoring, then ranked and threshold-selected output. SPECTER2 with its proximity

adapter produces 768-dimensional features. Separate bias-free projections map article and reaction features to 256 dimensions; the article projection is shared by global and sentence features. Reaction vectors are prepared once during initialization. The Python API, CLI and local HTTP service share the prediction implementation. Measurements use sequential local calls. Loading verifies file hashes and array shapes against the experiment manifest.

Metamorphic testing checks relations between transformed inputs and outputs when individual expected outputs are difficult to specify [4]. We apply this approach to candidate permutations, subsets and additions. The scoring equation determines the expected relations; the experiments test whether they hold in floating-point execution and across block configurations.

2.2. Scoring and execution contracts

Let g and s_j be the normalized projections of the global article and valid sentence features through the shared learned article projection, and r the normalized output of the separate learned reaction projection. The fixed head computes

$$z(x,r) = a\left[g^\top r + w\max\left(0, \max_j s_j^\top r - n\right)\right] + b, \quad \widehat{y}(x,r) = \mathbf{1}[z(x,r) \geq \tau]. \tag{1}$$

Here the learned scale a, local weight w, null n, bias b, and development-fitted threshold τ are held fixed. The implementation uses $a = \exp(\min(\ell, \log 100))$, $w = \mathrm{softplus}(u)$ and $n = \tanh(v)$ for learned scalars ℓ, u, v. Padding is masked before the sentence maximum. No reduction is taken over competing reactions. Consequently, in real arithmetic, permuting candidates permutes outputs; removing candidates preserves retained scores; and appending candidates preserves previous scores. Fixed-threshold decisions inherit this property. Rankings and top-k membership do not: new candidates can outrank existing ones. Updating descriptions, model weights, preprocessing or the threshold also falls outside this contract.

For batch size B, sentence count S, candidate count R and projected width P, local similarities require $O(BSRP)$ arithmetic. Candidate blocks of width K bound the principal local-similarity intermediate to $O(BSK)$ instead of $O(BSR)$. The final score matrix still uses $O(BR)$ storage, and the prepared reaction bank uses $O(RP)$ storage. Blocking organizes exact scoring into smaller intermediate tensors while preserving full catalog coverage.

2.3. Software functionalities

The local web interface accepts scientific text and displays ranked reactions and their scores. The CLI exports JSON or CSV predictions, and the Python

API and HTTP service use the same prediction path. Model downloads are explicit and checked against pinned hashes. The verification commands distinguish reconstruction of recorded tables, cached-feature inference and rerunning complete-text measurements. This separation lets a reader inspect the numerical evidence without downloading models and then repeat the inference or measurement workload when desired.

3. Illustrative examples

3.1. Local retrieval workflow

From a checkout with Python 3.11 or 3.12 and uv installed, a user runs `make setup` followed by `make serve`. The browser interface is available at `http://127.0.0.1:8767/`. One demonstration input is "STAT1 induces SMAD7 expression and inhibits TGF-beta signaling." The software scores the complete catalog and displays proposals for inspection. This example illustrates the retrieval workflow from input text to ranked catalog entries. The reviewer can inspect API documentation at `http://127.0.0.1:8767/docs`. Saving this demonstration sentence to a UTF-8 file named `abstract.txt`, the user can export the complete prediction payload with an empty title:

```
reaction-lens predict \
  --bundle artifacts/structured-seed23 \
  --encoder-dir artifacts/encoder \
  --title "" \
  --abstract-file abstract.txt \
  --format json --out predictions.json
```

The input file must be supplied by the user, and the output path must not already exist. The released README describes artifact preparation. Example results are proposals, and the scores are not calibrated biological probabilities.

In a single illustrative CPU run using the released structured seed23 epoch105 bundle, the software scored all 15,692 reactions. The first three returned reaction identifiers and raw logits were:

Rank	Reaction identifier	Raw logit
1	R-HSA-9730967	7.286
2	R-HSA-9736970	5.741
3	R-HSA-2106591	5.580

This output demonstrates full-catalog execution and the format of ranked results.

3.2. Reproducing the engineering evaluation

An AI assistant helped prepare measurement scripts and analyze results. I am responsible for the content.

The recorded evidence can be checked with `make verify`, without model downloads or third-party Python packages. After installation, `make check-model` repeats the cached-feature evaluation and `make benchmark` repeats the execution measurements. New observations are saved separately from the published record. The evaluation protocol is as follows.

Experiments use the fixed structured-description checkpoint from seed 23 at epoch 105 and its checksummed cached development features. Model parameters and the threshold are held fixed throughout the study. All 15692 catalog reactions are scored. We sample 64 publications without replacement using NumPy seed 20261005 and run CPU FP32 with two PyTorch threads. The environment is macOS 26.6.1 on arm64, Python 3.12.14, PyTorch 2.14.1 and NumPy 2.5.3. We recorded the arm64 architecture in the benchmark metadata.

Block widths are 128, 1024 and the full catalog (15692). Each block configuration receives five warmup queries and 3 timed repetitions per sampled publication. Timings exclude encoding, bundle loading, initial reaction projection and result serialization. Configurations run sequentially in fixed order. Medians and p95 values summarize the recorded repeated measurements.

On the first repetition we check candidate permutation, a half-catalog subset, appending 128 duplicate reaction vectors, and an unblocked reference. The predeclared absolute logit tolerance is 10^{-5}. The addition scenario reuses existing vectors, isolating candidate-axis execution from text encoding. Threshold checks use $\tau = 5.516382694244385$ without recalibration.

Memory is measured separately in a fresh process for each configuration using the same padded batch of 16 sampled publications, with maximum sentence count 14. We record process high-water resident memory including imports and loaded feature arrays. Each configuration has one memory trial. Processes used for this comparison do not execute the unblocked reference or the candidate-transformation probes.

Complete prediction measurements use 5 English title/abstract examples (PMIDs 10022914, 10047984, 10049780, 10067896 and 10069812) and three repetitions after one warmup. Prediction includes text encoding, scoring, ranking and payload construction. JSON serialization is recorded separately; HTTP/network costs are excluded. Model setup is timed separately.

4. Impact

The software makes a complete-catalog reaction retrieval implementation available for inspection and reuse, with an explicit distinction between changes in pairwise scores and changes in rank. It provides a starting point for investigating how catalog updates, candidate blocking and text representation affect scientific retrieval execution. These capabilities support reproducible comparisons of execution strategies and checks of implementation conformance to the specified scoring behavior.

The reproducibility package supports inspection of the implementation, reconstruction of the recorded tables and repetition of released-checkpoint inference. The measurements inform execution choices by separating text-encoding cost from full-catalog scoring and exposing the memory effect of candidate blocks.

4.1. Candidate and execution equivalence

The checks produce 10545024 scalar comparisons across 768 combinations of queries, configurations and scenarios. The maximum observed absolute logit error is 0.0, with 0 fixed-threshold flips. All observed errors satisfy the predeclared tolerance. The checks cover 64 publications, three block configurations and four scenarios, confirming the specified score and decision relations on the sampled inputs.

4.2. Blocking and local execution cost

Block width	Median (ms)	p95 (ms)	Peak RSS (MiB)
128	1.676	1.827	412.53
1024	0.642	0.701	414.17
15692	0.656	0.764	444.05

Table 2: Cached-feature single-publication scoring latency and separate padded-batch process memory. These are different workloads. Every timing row scores the full catalog. Latency covers 64 publications with three repetitions per configuration; memory uses one fresh-process trial per configuration.

The smallest block incurs greater observed latency. The default and unblocked configurations have similar observed medians. The blocked configurations have lower observed process peak RSS in the separate batch probe, consistent with the intermediate storage bound.

The 15 complete local predictions have median 268.90 ms and p95 393.54 ms. Encoding accounts for 96.33% of aggregate measured prediction time. Setup takes 4.87 s, and process peak RSS reaches 1149.22 MiB. For these examples,

improving only head execution offers limited benefit to total latency because encoding dominates. The component breakdown identifies encoding as the primary target for further latency experiments.

4.3. Inference consistency in a separate installation

We recomputed predictions in an isolated dependency environment using the downloaded checkpoint and cached features for 2377 development publications, 4094 known links and the full catalog. MAP is 0.4770, Hit@1 is 0.3925, and micro Recall@100 is 0.9016. Maximum absolute difference from the stored release metrics is 0.0.

For publication i, average precision sums the precision at each relevant rank over the full catalog and divides by the number of distinct recorded reaction labels for that publication, including labels absent from the catalog. MAP averages this value over publications. Hit@1 is the fraction of publications whose first result is recorded as relevant. Micro Recall@100 divides the total recorded links recovered in the first 100 results by all recorded links across publications. Equal logits retain catalog order. The absolute acceptance tolerance for the metric comparison is 5×10^{-5}.

The check recovers the recorded metrics from the fixed checkpoint and cached features. The complete-text latency experiment additionally exercises the local encoder. These are separate checks of cached-feature inference and complete-text execution.

4.4. Interpretation of results

The evaluation connects pairwise score stability, rank changes under catalog updates and execution cost. Candidate independence follows from the pointwise scoring equation; the experiments verify implementation conformance and quantify execution costs. The latency and memory measurements identify distinct optimization targets for text encoding and full-catalog scoring.

The reported timings characterize the workloads and environment specified in Section 3.2. Configurations were executed in fixed order; thermal and background-load effects were not controlled. RSS values are whole-process measurements. The unblocked reference uses the same scoring implementation, making the contract checks complementary to code inspection. The addition scenario isolates candidate-axis execution by reusing reaction vectors. The observed equivalence applies to the tested execution configurations.

Ranking metrics quantify agreement with recorded Reactome links. These labels are incomplete and were used during model development; evaluation on unseen publications is outside this execution study. The separate installation check verifies released-checkpoint inference using cached features, while the complete-text experiment exercises the local encoder.

5. Conclusions

Reaction Lens Systems connects a pointwise global/local scoring architecture to explicit execution contracts. On the measured inputs, candidate transformations and blocking preserve scores and fixed-threshold decisions. Blocking lowers observed process peak memory in the batch probe, while text encoding dominates complete local prediction time. The results provide a concrete procedure for checking candidate-axis behavior and separating the costs of exact catalog scoring from those of text representation.

Research materials

The implementation, experiment protocol and recorded measurements are available as research materials at revision `14c0ac048777`. Results in this article refer to that revision and the model and feature versions identified by its manifests.

The materials include contract checks, block measurements, component timings, the installation check, and metric definitions in code. The raw experiment record and numerical index connect aggregate values to observations. The protocol is described in Section 3.2; the accompanying protocol and execution instructions provide operational details for reconstructing tables, rerunning inference and repeating measurements. These are separate operations. Source checksums identify the measured scorer, encoder, pipeline and benchmark. Code is licensed under MIT and the manuscript under CC BY 4.0.

Author declarations

Funding: No external funding was received for this work.

Declaration of competing interests: I declare no competing financial interests or personal relationships that could have influenced this work.

CRediT author contribution statement: Imran Gadzhiev: Conceptualization, Methodology, Software, Investigation, Validation, Writing – original draft, Writing – review & editing.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

I used Codex (OpenAI) to assist with manuscript preparation under my direction. I take full responsibility for the content.

References

[1] E. Ragueneau, C. Gong, P. Sinquin, et al., The Reactome knowledgebase 2026, Nucleic Acids Research 54 (D1) (2026) D673–D681. doi:10.1093/nar/gkaf1223.
URL https://academic.oup.com/nar/article/54/D1/D673/8326464

[2] A. Cohan, S. Feldman, I. Beltagy, D. Downey, D. Weld, SPECTER: Document-level representation learning using citation-informed transformers, in: Proceedings of ACL, 2020, pp. 2270–2282. doi:10.18653/v1/2020.acl-main.207.

[3] A. Singh, M. D'Arcy, A. Cohan, D. Downey, S. Feldman, SciRepEval: A multi-format benchmark for scientific document representations, in: Proceedings of EMNLP, 2023, pp. 5548–5566. doi:10.18653/v1/2023.emnlp-main.338.

[4] S. Segura, G. Fraser, A. B. Sánchez, A. Ruiz-Cortés, A survey on metamorphic testing, IEEE Transactions on Software Engineering 42 (9) (2016) 805–824. doi:10.1109/TSE.2016.2532875.